



Universidad de Costa Rica
Facultad de Ingeniería
Escuela Ciencias de la Computación e Informática
Programación Paralela y Concurrente

Laboratorio de Algoritmos de Ordenamiento

Paralelización con OpenMP y Análisis con la Ley de
Amdahl

CI-0117

Grupo 01

Profesor:

Sleyter Angulo

Elaborado por:

Emanuel Hernández Arauz B93699

2026-05-10

Tabla de Contenidos

Abstract	2
1. Introducción	2
2. Marco teórico	2
3. Ambiente de ejecución	2
4. Metodología	3
5. Implementación	4
5.1. Odd-Even Sort	4
5.2. Shell Sort	5
5.3. Counting Sort	5
5.4. Algoritmos de División y Conquista	6
5.5. Bitonic Sort	6
6. Resultados	7
7. Análisis de resultados	7
8. Conclusiones	7
9. Referencias	8

Abstract

Este laboratorio presenta la implementación y análisis de la paralelización de cinco algoritmos de ordenamiento (Odd-Even Sort, Shell Sort, Counting Sort, Quick Sort, Merge Sort y Bitonic Sort) utilizando el framework OpenMP. La metodología se centró en el rediseño estructural de los algoritmos para exponer el paralelismo de datos y de tareas, incluyendo estrategias como el control de la granularidad mediante un umbral secuencial de 1024 elementos para mitigar el overhead de gestión de hilos. El análisis de rendimiento, basado en la Ley de Amdahl, reveló que la escalabilidad no se correlaciona directamente con la complejidad asintótica secuencial; algoritmos como Bitonic Sort demostraron una escalabilidad superior. Se concluye que el desafío crucial en la computación paralela es la coordinación eficiente de hilos y la optimización de la gestión de memoria y la granularidad, más que la reducción bruta de operaciones teóricas.

Keywords

Algoritmos de ordenamiento, Paralelización, OpenMP, Ley de Amdahl, Computación paralela, Granularidad, Escalabilidad.

1. Introducción

El manejo eficiente de volúmenes masivos de datos es un desafío central en la informática moderna. Si bien los algoritmos de ordenamiento secuenciales ofrecen una excelente complejidad asintótica, se convierten en cuellos de botella críticos cuando se ejecutan en máquinas con múltiples núcleos. El presente laboratorio aborda esta limitación mediante el estudio y la implementación de técnicas de paralelización.

El objetivo principal es explorar la viabilidad y el rendimiento de paralelizar cinco algoritmos de ordenamiento distintos (Odd-Even Sort, Shell Sort, Counting Sort, Quick Sort, Merge Sort y Bitonic Sort). Para ello, se emplea el framework OpenMP, reestructurando el código para explotar el paralelismo de datos y de tareas. Además, se busca realizar un análisis riguroso del *speedup* obtenido, contrastándolo con las predicciones teóricas de la Ley de Amdahl para comprender los límites de la escalabilidad en la práctica.

La importancia de esta investigación radica en demostrar que la eficiencia en la computación paralela depende de factores más allá de la complejidad secuencial. Se busca identificar las estrategias de diseño más robustas, como el control de la granularidad y la gestión de la memoria, que son esenciales para maximizar la eficiencia y el uso de los recursos en arquitecturas de memoria compartida.

1. Marco teórico

2. Ambiente de ejecución

Información detallada del sistema donde se ejecutaron las pruebas:

Procesador

Especificación	AMD Ryzen 5 3600 [zen 2 (Matisse)] - 6 núcleos, 12 hilos	
Litografía	7 nm (TSMC)	
Velocidad (MHz)	Base: 3600; Turbo: 4200 [x41.2 (22-36)] - Bus: 99.97	
Voltaje	1.100 V	
Caché	L1	6 x 32 kB (32 kB datos, 32 kB instrucciones), 8-way associative, 64-bytes (tamaño de línea) Velocidad: 124868.10 MB/s
	L2	6 x 512 kB, 8-way associative, 64-bytes (tamaño de línea) Velocidad: 121066.90 MB/s
	L3	2 x 16 MB, 16-way associative, 64-bytes (tamaño de línea) Velocidad: 69587.10 MB/s
Instrucciones	SMT, MMX(+), SSE(1, 2, 3, 3S, 4.1, 4.2, 4A), AVX(1, 2), FMA(3), AES, CLMUL, RdRand, SHA, AMD-V, x86-64	
Memoria		
Tipo	DIMM DDR4 (Nº de parte: MRX4U320NNNF16G)	
Canales	Canal A 16 Gb; Canal B 16 Gb - Tamaño total: 32 Gb (Dual Channel) (Synchronous Unbuffered)	
Especificación	Voltaje: 1.2 V Velocidad: 3200 MT/s (config.); 3200 MT/s (max.)	
Sistema Operativo		
Nombre	Ubuntu 24.04.4 LTS (server)	
Especificación	Hardware físico Kernel: Linux 6.17.0-23-generic Compilador: GCC versión 14.2.0 (Ubuntu 14.2.0-4ubuntu2~24.04.1) OpenMP: versión 201511	

Optimizaciones adicionales: Se utilizó la distribución de Ubuntu Server para evitar bloatware y cargar al sistema de herramientas innecesarias que no están presentes en servidores (interfaz de usuario y todos los servicios de fondo que conlleva) y se redujo al montaje del servidor X11, gestor de ventanas i3wm, Network Manager, vim, yazi, picom, feh, polybar, udiskie y dunst para tener una experiencia de usuario decente y completa mas

no inmersiva.

3. Metodología

El proceso de implementación se abordó desde un análisis profundo de las dependencias de datos inherentes a cada algoritmo. La estrategia general no consistió simplemente en inyectar directivas de OpenMP en los bucles existentes, sino que requirió rediseñar estructuralmente varios algoritmos para exponer su nivel de paralelismo sin comprometer la integridad de los datos.

Una de las decisiones de diseño más importantes a lo largo del proyecto fue el control de la granularidad mediante la definición de un umbral secuencial (cut-off). Durante las pruebas iniciales, observamos que algoritmos recursivos como Quick Sort y Merge Sort sufrían una degradación de rendimiento severa si se permitía que OpenMP creara tareas para subarreglos muy pequeños. El costo operativo (overhead) de gestionar los hilos superaba el tiempo de cómputo útil. Para mitigar esto, se utilizó una condición de salida que redirige el flujo hacia una versión puramente secuencial del algoritmo cuando el tamaño del problema se reduce a menos de 1024 elementos (umbral de granularidad), optimizando así el balanceo de carga y el uso de la caché del procesador.

Otra decisión crítica a nivel arquitectónico fue la gestión de memoria. En algoritmos como Merge Sort o Counting Sort, que requieren memoria auxiliar, se optó por reservar los arreglos temporales una única vez en el bloque principal (main) y pasar sus referencias por parámetro. Esto evitó cuellos de botella generados por múltiples hilos intentando acceder al gestor de memoria del sistema operativo (malloc/free) de forma simultánea.

En cuanto a la estrategia de paralelización, los algoritmos se agruparon según su viabilidad arquitectónica:

Paralelismo de Datos (Bucles): Aplicado en algoritmos con iteraciones independientes como *Odd-Even Sort* y *Bitonic Sort*.

Paralelismo de Tareas (Divide y Vencerás): Utilizado en *Merge Sort* y *Quick Sort*, delegando subproblemas a una piscina de hilos (*task pool*).

Reducciones Concurrentes: Empleado en *Counting Sort* para evitar condiciones de carrera al manipular histogramas compartidos.

Paralelismo Dinámico: En *Shell Sort*, donde la cantidad de hilos útiles disminuye conforme avanza el algoritmo.

Para el protocolo de medición, nos enfocamos en aislar el tiempo de cómputo real. Se utilizó la función `omp_get_wtime()` midiendo exclusivamente las regiones de ordenamiento (excluyendo la inicialización de datos y validaciones). Para garantizar fiabilidad, cada algoritmo se ejecutó un mínimo de cinco veces consecutivas bajo la misma configuración, promediando los resultados para mitigar anomalías provocadas por procesos en segundo plano del sistema operativo. Las pruebas de escalabilidad se estructuraron variando la cantidad de hilos (1, 2, 4, 8 y 12) mediante la cláusula `num_threads` para observar el impacto real de la Ley de Amdahl en cada enfoque.

4. Implementación

A continuación, se detalla el tratamiento que recibió cada algoritmo durante la fase de codificación, junto con la justificación de las directivas OpenMP seleccionadas.

5.1. Odd-Even Sort

Dado que Bubble Sort es secuencial debido a la dependencia de sus intercambios, preferimos utilizar Odd-Even Sort para facilitar la paralelización y garantizar una mejora sustancial en el rendimiento. La paralelización se basó en dividir el algoritmo en dos fases (pares e impares) que garantizan que ningún hilo evalúe índices superpuestos.

```
#pragma omp parallel default(none) shared(arr, n) private(i, temp, fase)
{
    for (fase = 0; fase < n; fase++) {
        if (fase % 2 != 0) {
            #pragma omp for
            for (i = 1; i <= n - 2; i += 2) {
                if (arr[i] > arr[i + 1]) { /* Intercambio */ }
            }
        } else {
            #pragma omp for
            for (i = 0; i <= n - 2; i += 2) {
                if (arr[i] > arr[i + 1]) { /* Intercambio */ }
            }
        }
    }
}
```

El bloque `#pragma omp parallel` se colocó estratégicamente fuera del bucle de fases. De esta manera, el equipo de hilos se crea una sola vez, eliminando la sobrecarga de instanciar hilos en cada iteración. Dentro, usamos `#pragma omp for` para repartir las comparaciones. Aprovechamos la barrera de sincronización implícita al final de cada bloque `for` para garantizar que la fase impar termine completamente antes de que cualquier hilo inicie la fase par.

5.2. Shell Sort

El reto principal con *Shell Sort* fue la inanición de hilos (*thread starvation*). Se optó por reestructurar el código de dos a tres bucles anidados para hacer evidentes los subarreglos independientes ante OpenMP.

```
for (gap = n / 2; gap > 0; gap /= 2) {
    #pragma omp parallel for private(j, k, temp) shared(arr, n, gap)
    for (i = 0; i < gap; i++) {
```

```

    for (j = i + gap; j < n; j += gap) {
        // Lógica de Insertion Sort sobre el subarreglo actual
    }
}
}

```

El bucle exterior (el cálculo del *gap*) debe ser secuencial, pero para un salto dado, existen exactamente *gap* subarreglos que no se tocan entre sí. Usamos `#pragma omp parallel for` en el segundo bucle para distribuir estos subarreglos. Variables como *j*, *k* y *temp* se declararon como `private` para evitar que los hilos sobrescriban sus índices de trabajo locales.

5.3. Counting Sort

Este algoritmo representaba un riesgo alto de condiciones de carrera durante la creación del histograma de frecuencias, ya que múltiples hilos podrían intentar incrementar la misma celda de conteo al mismo tiempo.

```

// Conteo paralelo usando reducción de arreglos
#pragma omp parallel for reduction(+:count[0:k+1])
for (int i = 0; i < n; i++) {
    count[arr[i]]++;
}

```

En lugar de usar secciones críticas (`#pragma omp atomic`), que habrían destruido el rendimiento, se utilizó la cláusula `reduction` sobre arreglos introducida en OpenMP 4.5+. Esto fuerza al compilador a generar una copia privada del arreglo `count` (llena de ceros) para cada hilo. Al finalizar el bucle, OpenMP suma automáticamente los arreglos locales en el arreglo global de forma segura y sumamente rápida.

5.4. Algoritmos de División y Conquista

Para estos algoritmos, el paralelismo de bucles simplemente no sirve - ya que no utilizan estos, al menos para la sección de ordenamiento - por lo que se optó por el uso del modelo de tareas (*Task Parallelism*) que se adapta de forma natural a la recursividad.

```

// Fragmento de Quick Sort Paralelo
if ((high - low) < SEQ_THRESHOLD) {
    quick_sort_s(arr, low, high);
    return; // Condición de salida temprana
}

```

```
int pi = partition(arr, low, high);
```

```
#pragma omp task shared(arr) firstprivate(low, pi)
```

```

quick_sort_p(arr, low, pi - 1);

#pragma omp task shared(arr) firstprivate(pi, high)
quick_sort_p(arr, pi + 1, high);

#pragma omp taskwait

```

Usamos `#pragma omp task` para encolar cada mitad del arreglo como un trabajo independiente. Es vital el uso de `firstprivate` para capturar el valor exacto de los límites (`low`, `high`, `pi`) en el momento en que se crea la tarea, evitando que la recursividad posterior los altere. La directiva `#pragma omp taskwait` detiene la ejecución del nodo padre hasta que sus tareas hijas finalicen.

Para lanzar estos algoritmos desde el `main`, se utilizó la directiva `#pragma omp single nowait`. Esto asegura que un único hilo lance la tarea raíz, mientras el `nowait` libera inmediatamente al resto de hilos para que empiecen a procesar las tareas que se van generando en la cola.

5.5. Bitonic Sort

Al ser una red de ordenamiento, *Bitonic Sort* carece de dependencias de datos ligadas a los valores del arreglo, lo que lo hace el algoritmo más óptimo para el paralelismo.

```

if (cnt > SEQ_THRESHOLD) {
    #pragma omp parallel for
    for (int i = low; i < low + k; i++) {
        if (dir == (arr[i] > arr[i + k])) {
            swap(&arr[i], &arr[i + k]);
        }
    }
} else {
    // Bucle secuencial sin overhead
}

```

A diferencia de *Quick Sort* donde las tareas envuelven a la función entera, aquí el trabajo pesado (la mezcla) recae en un bucle `for` simple totalmente determinista. Usamos `#pragma omp parallel for` para dividir las comparaciones masivas, condicionando su creación al tamaño del bloque evaluado. Al no haber posibilidad matemática de que dos hilos crucen índices, no se requieren directivas de sincronización complejas ni variables privadas extra.

5. Resultados

Presentación de los datos recopilados:

- Tablas con tiempos de ejecución (promedio y desviación estándar)
- Gráficos de tiempo vs. tamaño de arreglo (para cada algoritmo)

- Gráficos de speedup real vs. speedup teórico (Ley de Amdahl)
- Gráficos de eficiencia vs. número de hilos
- Gráficos comparativos entre algoritmos

6. Análisis de resultados

Interpretación de los datos, discusión sobre el speedup obtenido, comparación con la predicción de Amdahl, explicación de anomalías, análisis de la eficiencia y escalabilidad.

7. Conclusiones

Tras el desarrollo y análisis de las implementaciones en OpenMP, se desprenden los siguientes hallazgos y lecciones fundamentales sobre la computación paralela aplicada al ordenamiento de datos:

1. Hallazgos principales y la "Paradoja de la Complejidad"

Uno de los hallazgos más reveladores es que la eficiencia de un algoritmo en paralelo no siempre guarda una relación directa con su complejidad asintótica secuencial. Mientras que Quick Sort y Merge Sort siguen siendo los líderes en rendimiento práctico debido a su trabajo base de $O(n \log(n))$, algoritmos como Bitonic Sort $O(n \log^2(n))$ demostraron una escalabilidad superior en entornos con alta disponibilidad de hilos. Esto nos demuestra que, en el ámbito paralelo, la predictibilidad del flujo de datos y la ausencia de *branches* condicionales suelen ser más valiosas que el conteo bruto de operaciones teóricas.

2. Costo de la Abstracción

La implementación evidencia que el paralelismo no es gratis. El concepto de overhead o sobrecarga de gestión es el factor determinante entre una ganancia de velocidad real y una degradación de rendimiento. Aprendimos que la Ley de Amdahl no es solo una fórmula teórica, sino una barrera física: por muy rápido que sea el conteo en Counting Sort o la partición en Quick Sort, si la gestión de hilos o las secciones secuenciales (como el volcado de datos) no están optimizadas, el beneficio de tener múltiples núcleos se diluye rápidamente.

3. Análisis sobre la Mezcla Paralela (CLRS)

Basándonos en la literatura de Introduction to Algorithms (Cormen et al.), exploramos la posibilidad de paralelizar incluso el procedimiento de mezcla (merge) en el *Merge Sort* para eliminar el cuello de botella secuencial de $O(n)$. Si bien la teoría indica que esto reduce el span del algoritmo a $O(\log^3(n))$, la implementación práctica nos demostró que es un arma de doble filo. La lógica necesaria para realizar búsquedas binarias cruzadas y coordinar múltiples tareas dentro de una misma mezcla genera una complejidad de gestión tan alta que, en arreglos de tamaño pequeño o mediano, se produce una penalización de tiempo notable.

La ganancia solo se vuelve real en volúmenes de datos masivos que superan las capacidades de la memoria caché convencional.

4. Recomendaciones para la paralelización: Para futuros desarrollos en el área, se recomiendan las siguientes directrices:

- Priorizar la granularidad: El uso de umbrales se vuelve una necesidad para proteger al procesador de la saturación por tareas diminutas.
- Optimizar el acceso a memoria: El diseño debe centrarse en la localidad de los datos. El uso de memcpy para operaciones en bloque y la reutilización de arreglos temporales

(evitando malloc recurrentes) tienen un impacto más significativo en el tiempo final que la optimización de los propios bucles de comparación.

- Evaluar el escenario de uso: Como dice mi querida profesora Sandra Kikut V. “no existe **el** mejor algoritmo de ordenamiento”. Para sistemas con memoria compartida limitada, *Quick Sort* sigue siendo la opción más eficiente; para sistemas masivamente paralelos o hardware específico, arquitecturas como Bitonic Sort ofrecen un techo de rendimiento mucho más alto.

En conclusión, la programación paralela con OpenMP nos obliga a dejar de pensar únicamente en términos de "pasos de ejecución" para empezar a pensar en términos de flujos de trabajo, sincronización y congruencia, donde el reto real no es solo procesar más rápido, sino coordinar de forma inteligente y - volviendo a citar a Kikut - “eficaz y eficiente”.

8. Referencias

[1] GitHub. (s.f.) *TheTumultuousUnicornOfDarkness/CPU-X: CPU-X is a Free software that gathers information on CPU, motherboard and more*. Disponible en: <https://github.com/TheTumultuousUnicornOfDarkness/CPU-X> (accesado: 10 mayo 2026).

[2] OpenMP. (s.f.) *OpenMP Compilers & Tools*. Disponible en: <https://www.openmp.org/resources/openmp-compilers-tools/> (accesado: 10 mayo 2026).

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). The MIT Press.

[4] Lopera, J. O. (2009). *Entre la profecía de Moore y la ley de Amdahl*.

[5] Vásquez Arguello, O. J. (2014). *Escalabilidad y desempeño en clústers*.

[6] GeeksforGeeks. (s.f.). *Data Structures and Algorithms (DSA)*. <https://www.geeksforgeeks.org/dsa/dsa/> (Accedido el 11 de mayo de 2026).

[7] W3Schools. (s.f.). *DSA Tutorial*. <https://www.w3schools.com/dsa/> (Accedido el 11 de mayo de 2026).

[8] Programiz. (s.f.). *Learn Data Structures and Algorithms*. <https://www.programiz.com/dsa> (Accedido el 11 de mayo de 2026).

[9] Tutorialspoint. (s.f.). *Data Structures and Algorithms Tutorial*. https://www.tutorialspoint.com/data_structures_algorithms/ (Accedido el 11 de mayo de 2026).